Assignment: Flask Application with Git Versioning Workflow

What is This Assignment About?

This assignment has two goals running in parallel:

The first is to build a small web application using Python and Flask, where different URLs return different responses just like how amazon.com/orders shows your orders and amazon.com/cart shows your cart. Each URL in your app will do one specific job.

The second is to track your work using Git, saving your progress in stages called versions. This is the same way professional developers manage real software projects: nothing goes live without being tested in a separate branch first.

By the end, your project must be hosted on GitHub with a clear history showing how your app evolved from Version 1 to Version 2.

Objective

The objective of this assignment is to help learners:

- Build a basic web application using Python (Flask)
- Understand what REST API endpoints are and how they work
- Practice Git branching and version control workflows
- Implement feature-based development across multiple versions
- Document and present their work professionally using GitHub

Prerequisites

Before attempting this assignment, learners should be familiar with:

- Basic Python programming (functions, dictionaries, return statements)
- Installing and running Python files from the terminal
- Basic Git commands (clone, add, commit, push)
- Creating and managing Git branches
- What a GitHub repository is and how to create one

Tools Required

- Python 3.x
- Flask
- Git
- GitHub account
- Code editor (VS Code recommended)

Assignment Overview

You are required to:

- Build a Flask-based web application
- Use Git branching (dev and main) for version control
- Implement features across two versions
- Maintain proper documentation in a README file

Git Workflow Requirement (Mandatory)

You must follow this workflow strictly:

- Initialize a Git repository in your project folder
- Create a branch named dev
- Do all development work inside the dev branch
- Merge dev into main only when the feature is complete and working
- Repeat this process for each version

This means main should only ever contain stable, working code. The evaluator will check your branch and merge history to verify this.

Tasks

Task 1: Basic Flask Application

What you need to build: A Python file that runs a local web server with two working endpoints.

Endpoint	Expected Response
/	Welcome to the App
/health	App is running

How to verify your work: Visit http://localhost:5000/ and http://localhost:5000/health in your browser. Both should show the correct text.

Ensure:

- The application runs successfully on localhost
- All endpoints return the correct response and are accessible via browser or Postman

Task 2: Git Setup and Version 1 Release

What you need to do: Set up proper Git version control for your project and publish your Task 1 code to GitHub as Version 1.

Steps to follow in order:

1. Initialize a Git repository inside your project folder
2. Create a branch named dev and switch to it
3. Add your files and commit them with a descriptive commit message
4. Push the dev branch to your GitHub repository
5. Merge dev into main
6. Push main to GitHub

Task 3: Feature Implementation

Choose ONE of the following applications and implement the required endpoints.

Option A: Voting Application (Beginner Level)

What you are building: A simple in-memory voting system where users can cast votes for candidates by visiting a URL and view the current standings.

Endpoints to implement:

Endpoint	Behaviour
/vote/<name>	Records one vote for the candidate with that name. If they already have votes, the count increases. If they are new, their count starts at 1.

/results	Returns the current vote count for all candidates in JSON format

What you need to figure out:

- How to use a variable segment in a Flask route so the name in the URL becomes a value your function can use
- How to store and update vote counts in memory using a Python dictionary
- How to handle a candidate name that does not exist in your dictionary yet
- How to return data as JSON from a Flask endpoint

Think about: What data structure will hold your votes? What happens the very first time someone votes for a name that has never been seen before?

Option B: Password Manager (Intermediate Level)

What you are building: A basic in-memory store where users can save a username and password, and retrieve the password later by username.

Endpoints to implement:

Endpoint	Method	Behaviour
/add	POST	Accepts a JSON body with a username and password field and stores them
/get/<username>	GET	Returns the stored password for that username, or an appropriate error if the username does not exist

What you need to figure out:

- Why /add must be a POST request and not GET — understand the difference before implementing
- How to read JSON data from the body of an incoming POST request in Flask
- How to respond with an error when a username is not found
- Why test a username that was never added?

Ensure:

- Data is stored in memory (a dictionary is sufficient — no database needed)
- Proper request handling is implemented for all cases

Task 4: Version 2 Enhancement

What you need to do: Add one new endpoint to your existing application based on the option you chose, then release it as Version 2 using the same Git workflow from Task 2.

For Voting Application — add /reset: This endpoint should clear all stored vote counts and return a confirmation message. After calling it, /results should show no data.

For Password Manager — add /delete/<username>: This endpoint should remove the specified user's record from storage and confirm deletion. If the username does not exist, return an appropriate error message.

Git steps required:

- All new development must happen in the dev branch — do not add code directly to main
- Commit with a clear message describing the new feature
- Push dev to GitHub
- Merge dev into main
- Push main to GitHub

The evaluator will verify that your Git history shows Version 2 was built on top of Version 1, not rewritten from scratch.

Ensure:

- The new endpoint works correctly
- The Git history clearly shows a second merge from dev into main

Task 5: Documentation

What is a README? It is the first thing anyone sees when they visit your GitHub repository. A good README means someone who has never seen your code can understand what it does, set it up on their machine, and use it — without asking you a single question.

Your README.md must include the following sections:

Project Title and Description What does your app do? Write 3 to 4 lines explaining it in plain English. No technical jargon.

Installation and Setup Steps Step-by-step instructions for how someone else can download and run your app. Include every command they would need to type, in the correct order.

API Endpoint Reference A table listing every endpoint, its URL, the HTTP method, what it does, and an example of the response it returns.

Git Workflow Explain in your own words how you used dev and main branches in this project. A simple diagram or a written description of the flow is acceptable.

Version History A short table or list describing what was included in Version 1 and what was added in Version 2.

Screenshots (mandatory) The following screenshots must be embedded directly inside the README file — not attached separately:

- The application running in a browser showing at least one working endpoint
- The GitHub repository page showing both the dev and main branches
- The commit or merge history showing the Version 1 and Version 2 releases

Common Mistakes to Avoid

Mistake	Why It Costs Marks
Only using the main branch with no dev branch	The entire Git workflow section cannot be evaluated
Vague commit messages such as "fix" or "done"	Commit quality is part of the Git evaluation criteria
Endpoints not returning JSON where required	Functionality marks are deducted
README missing sections or screenshots	All 5 documentation marks are at risk
Application throws errors when run	Affects evaluation across multiple sections

Expected Outcome

By completing this assignment, learners will:

- Understand how web servers respond to different URL requests
- Know what REST APIs are and how they work in practice
- Be comfortable with the Git branching workflow used in professional teams
- Understand what software versioning means and how to implement it
- Be able to write documentation that others can follow independently